

Metaheurísticas

Seminario 2. Problemas de optimización con técnicas basadas en búsqueda local

Problema del Portolio

1. Agrupamiento Clásico
2. Problema del Portolio: Definición y Variantes del Problema
3. Análisis del Problema
4. Evaluación de Soluciones
5. Generación y Reparación de Soluciones
6. Solución Greedy
7. Búsquedas por Trayectorias Simples
8. Casos del problema

Problema del Portfolio



*¿En qué acciones invertir
nuestro dinero?*

- Queremos:
 - Rentabilidad.
 - Diversificar.
 - Reducir Riesgo.



A. Salo, M. Doumpos, J. Liesiö, y C. Zopounidis, «Fifty years of portfolio optimization», European Journal of Operational Research, vol. 318, n.o1, pp. 1-18, oct. 2024, doi: [10.1016/j.ejor.2023.12.031](https://doi.org/10.1016/j.ejor.2023.12.031).

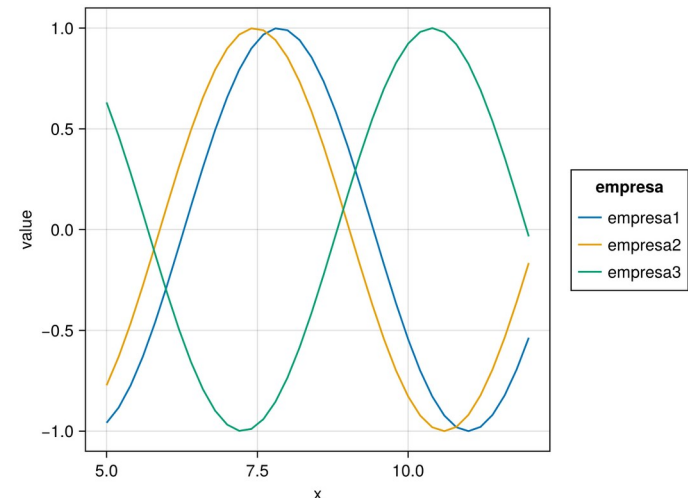
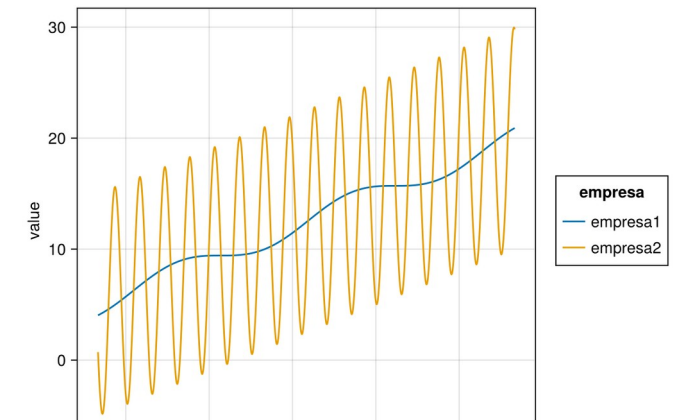
Es un problema multiobjetivo

- Beneficio, medido en unidades (días, semanas, ...).



- Riesgo

- Covarianza o Correlación.



Múltiples versiones

- Versión clásica (Markowitz, 1952)

Minimizar $w^T \Sigma w$ $\mu^T w \geq \mu^*$, $1^T w = 1, w_i \geq 0 (i=1, \dots, n)$.

Σ : matriz de covarianzas.

μ : vector de retornos esperados.

μ^* : retorno objetivo.

- Fácil de abordar, convexo.
- Muy poco flexible, exige un retorno esperado.

Múltiples versiones

- Media-varianza con aversión al riesgo

Maximizar $w\mu^\top w - \lambda w^\top \Sigma w, \lambda > 0$.

- No exige el retorno.
- Sin restricciones

Múltiples versiones

- Media-varianza con restricciones (el más complejo)

Minimizar $w^T \Sigma w$ sujeto a:

$$\mu^T w \geq \mu^*$$

$$1^T w = 1$$

$$w_i \in \{0\} \cup [w, w]$$

$$\sum w^{\{w_i > 0\}} \leq K,$$

- Exige solo K elementos.
- Mantiene un rango.

Nuestra versión

- Nuestra versión

Maximizar $w\mu^\top w - \lambda w^\top \Sigma w, \lambda > 0$, sujeto a:

$$1^\top w = 1$$

$$w_i \in \{0\} \cup [w, w]$$

- Combinamos linealmente el beneficio y riesgo.
- No planteamos un beneficio mínimo.
- Exigimos solo el rango, no un número de empresas.

Es un problema con restricciones



- Inversión máxima por acción.
- Inversión mínima por empresa (siempre puede ser cero).
- Se puede limitar el número de empresas.

Análisis del problema

- Una solución es un vector de valores entre 0 y `max_value`.
- Los valores inferiores a `min_val` serán todos 0.
- Se combinan los objetivos linealmente según un valor `lambda`.
- Aplicaremos valores reales.

Datos de la Práctica

- Se suele usar benchmarks de portfolio, con datos históricos ya procesados.
- A menudo agrupan datos de trimestres como si fuesen empresas distintas.
- Vamos a usar nuestros propios datos reales (Fuente: Yahoo Finances):
 - IBEX-35: Bolsa española.
 - S&P-500: 500 principales empresas de EEUU.
 - S&P-100: 100 principales empresas de EEUU (subconjunto del anterior).
- Periodo 2015-2025, años completos, guardados diariamente (los días depende de la Bolsa).

Datos de la Práctica

- Distinguimos periodo histórico (2015-2024) del reciente (2025).
- Vamos a optimizar los mejores pesos para el periodo histórico, y valorar soluciones.
- La mejor solución obtenida se evaluará una vez para el 2025, solo para ver qué pasaría.

Formato de los datos: CSV

- Están en ficheros CSV, con el formato siguiente:
 - Date, Emp1, Emp2,, EmpN
- Almacena para cada día la mejora relativa (en porcentaje) de invertir en dicha empresa.
- No hay valores perdidos, se han eliminado empresas que no perdurasen todo el periodo.
- Datos fáciles de leer y procesar.

Algoritmo Aleatorio

- Escoger aleatoriamente un número de zeros, `nzeros`.
- Asignar `nzeros` ceros a posiciones aleatorias.
- Para el resto generar elementos entre `min_value` y `max_value`.
- Reparar la solución.
- ¿Por qué hay que reparar?
 - Es fácil que la suma de pesos sea 1, pero más difícil con los rangos.
 - Si $\text{suma}(w) \neq 1$, $w = w / \text{suma}(w)$.
 - Eso hace que se puedan violar los límites.

Reparación de Soluciones

- Los límites máximos y mínimos complican la búsqueda (sobre todo el mínimo).
- Es fácil que la suma de pesos sea 1, pero más difícil con los rangos.
- Proceso:
 - Si $\text{suma}(w) \neq 1$, $w = w / \text{suma}(w)$.
 - Si $w_i > \text{max_value}$, $w_i \leftarrow \text{max_value}$
 - Si $w_i < \text{min_value}$, $w_i \leftarrow 0$
 - $\text{Dif} \leftarrow 1 - \text{suma}(w)$
 - Mientras $\text{Dif} > 0$
 - Escoger pesos que se puedan incrementar (solo usar aquellas con valor 0 si es necesario).
 - Dividir la diferencia entre ellos, sumando un valor (comprobar que puedan).
 - $\text{Dif} \leftarrow 1 - \text{suma}(w)$

Algoritmo Greedy

- Hay muchas versiones, muchas tiene poco de *greedy*.

- Aplicaremos el esquema más directo:

$$Heurística(E_i) = \frac{\sum_t \log(1 + E_i)}{T} + \lambda \cdot \frac{\sum_j Cov(E_i, E_j)}{N}$$

- Calcular las medias de beneficio.
- Calcular para cada empresa la media de covarianza con el resto.
- Ordenar las empresas por su heurística, y escoger la mayor cantidad posible:

$$W(i) = \max_{value} si \sum (w) + \max_{value} \leq 1, 1 - \sum (w) \text{ en otro caso}$$

Búsqueda por Trayectorias Simples

El método de búsqueda por trayectorias simples que usaremos es la **Búsqueda Local**

Para construir un esquema de optimización por búsqueda local tenemos que definir la **función objetivo y el operador de vecino**, así como el esquema de representación del problema

Búsqueda por Trayectorias Simples: El Algoritmo

El algoritmo propuesto utiliza una estrategia tipo “**el primer mejor**” para explorar el entorno de una solución. Esto es, se acepta el primer vecino que mejore a la solución actual como nueva solución

La exploración del entorno no se hace en un orden determinístico sino en un orden aleatorio para dar diversidad

Para evaluar la calidad de las soluciones utilizamos la función objetivo f , que hay que minimizar. De esta forma se acepta como nueva solución el primer vecino S' que cumpla $f(S') < f(S)$

El proceso termina cuando se alcanza el número máximo de evaluaciones de la función objetivo o cuando no se produce mejoría al explorar un vecindario concreto ($\nexists S' t. q. f(S') < f(S)$)

La solución inicial se genera de manera aleatoria, comprobando que cada cluster tiene al menos una instancia

Búsqueda por Trayectorias Simples: El Algoritmo

El algoritmo propuesto utiliza una estrategia tipo “**el primer mejor**” para explorar el entorno de una solución. Esto es, se acepta el primer vecino que mejore a la solución actual como nueva solución

La exploración del entorno no se hace en un orden determinístico sino en un orden aleatorio para dar diversidad

Para evaluar la calidad de las soluciones utilizamos la función objetivo f , que hay que minimizar. De esta forma se acepta como nueva solución el primer vecino S' que cumpla $f(S') < f(S)$

El proceso termina cuando se alcanza el número máximo de evaluaciones de la función objetivo o cuando no se produce mejoría al explorar un vecindario concreto ($\nexists S' t. q. f(S') < f(S)$)

La solución inicial se genera de manera aleatoria, comprobando que cada cluster tiene al menos una instancia

Búsqueda por Trayectorias Simples: Operador de Vecino

- El operador de vecino aplicado implica el intercambio de una cantidad entre una empresa y otra.
- Dada una solución se escoge un índice (instancia x_i) y se cede parte de su cantidad a un índice j (instancia x_j).
- La cantidad a mover es un 40% de la cantidad inicial.
- Se comprueban los límites.
- No es necesario comprobar la suma, ya que se mantiene.

Búsqueda por Trayectorias Simples: Operador de Vecino

- El entorno de la solución s está formado por las soluciones accesibles desde ella a través de la aplicación de dicho operador, sobre todos los índices de S , $i \in \{1, \dots, n\}$.
- El tamaño del entorno de una solución es $n(n-1)$
- No necesitamos generar todo el entorno.1.
 - 1. Generar todo el conjunto P de pares (i,j) con $i \neq j$.
 - 2. Se baraja de forma aleatoria: $P = \text{shuffle}(P)$.
 - 3. Para cada p en P :
 - 4. $\text{nuevo} = p$; $\text{nuevo}[j] += p[i]*0.4$; $\text{nuevo}[i] -= 0.4*p[i]$.
 - 5. Si $\text{fitness}(p) < \text{fitness}(\text{best})$ otra iteración.
 - 6. Si $\text{fitness}(p) > \text{fitness}(\text{best})$ actualiza best , y vuelve al paso 2.
 - 7. Si nos quedamos sin pares, el algoritmo para (*óptimo local*).

Muchas gracias



Daniel Molina Cabrera
dmolinac@ugr.es
<https://www.danimolina.net>